

WEB103 | Advanced Web Development

Advanced Web Development Summer 2026 (@ Section 1 | Wednesdays 5PM - 7PM PT)

Personal Member ID#: 129018

Need help? Post on our [Class Slack Channel](#) or email us at support@codepath.org

Getting Started

Learning with AI ✨

► Course Info

Course Progress

▼ Individual

Unit 1

Unit 2

Unit 3

Unit 4

► Capstone Group Project

Overview

Lab

Projects

IDE Setup

Learning with AI ✨

Resources

Report

Unit 1 Lab: UnEarthed, Part 1

Overview

What should you get for someone who has everything (can't relate)? What about someone who hates stuff? [Unearthed](#) has your answer: it's a crowdsourced repo of great gift ideas for different kinds of people. Browse gifts by intended audience, price point, or category, or upload your own gift ideas!

[Cat_Disguised_as_a_Guy_Named_Fred](#) just submitted a great gift for Gamers: A \$\$ (two-dollar sign) headset with a great image and description! Thanks, [Cat_Disguised_as_a_Guy_Named_Fred](#)! You really

saved my `[insert holiday here]`. In the first part of this four-part lab, you'll create the MVP (minimum viable product) for `Unearthed`: a site that features static HTML with only a few finds tailored towards certain audiences.

Goals

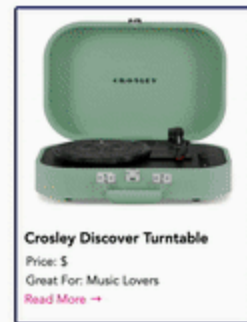
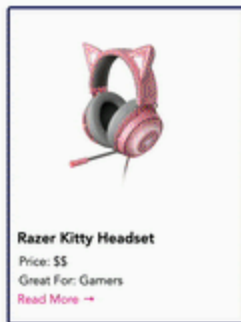
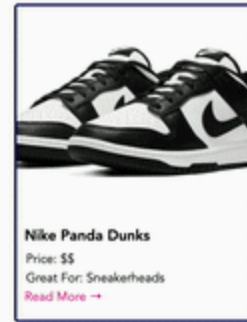
By the end of this lab you will be able to...

- ☐ Create a server using Node and Express
- ☐ Create routes and request handlers using Express
- ☐ Respond to a `GET` request by serving static HTML documents
- ☐ Serve an appropriate document in the case of no matching route
- ☐ Organize server files appropriately

Features

Required Features

- ☐ The web app uses only HTML, CSS, and JavaScript without a frontend framework.
- ☐ The web app displays the title and logo of `Unearthed`.
- ☐ The web app displays a list of gifts that have been submitted, including at least the name, price point, audience, and image.
- ☐ The user can click on each item in the list to see a detail view of it, including all database fields.
- ☐ The web app serves an appropriate 404 page when no matching route is defined.



Stretch Features

- ☐ The web app is appropriately styled using Picocss

Resources

- [MDN Web Docs: HTML](#)
- [MDN Web Docs: CSS](#)
- [Picocss](#)
- [MDN Web Docs: JavaScript](#)
- [Express Routing](#)
- [Express Basic Routing](#)

Lab Instructions

- 👉 Before proceeding, make sure you've set up your development environment by following the [IDE Setup Guide!](#)

Required Features

Step 0: Set up a Node.js project

In this step, we'll initialize the Node.js project and install the necessary dependencies.

- ☐ Download and install the [most current version of Node.js](#).
- ☐ Create a new GitHub repository for this lab called `unearthed`.
- ☐ Clone the repository and open it in VS Code.

Inside the `unearthed` directory, set up the client:

- ☐ Using the terminal, run the command `npm create vite@latest`.
- ☐ Enter `client` as the **Project name**.
- ☐ Select `Vanilla` as the framework.
- ☐ Select `JavaScript` as the variant.
- ☐ Run the command `cd unearthed`, then run the command `npm install` to install dependencies.

Next, let's set up the server:

- ☐ Inside the `unearthed` directory, create a new directory called `server`.
- ☐ Using the terminal, navigate into the `server` directory using `cd server`.
- ☐ In the `server` directory, run the command `npm init -y`. This initializes the project and creates a `package.json` file, which enables `npm` to start your project, run scripts, install dependencies.
- ☐ In the `server` directory, run the command `npm install express nodemon` to install the `express` and `nodemon` packages.

💡 When an `express` server is running, any changes we make to the code do **not** automatically reset the server so we can see the changes. However, we can use the `nodemon` package, which allows us to automatically reset the server when we make changes.

In the `server\package.json` file:

- ☐ Change the value of the `"main"` field from `"index.js"` (line 5) to `"server.js"`.
- 💡 Setting the `"main"` value in the `package.json` file to `"server.js"` tells Node.js that the `server.js` file is the primary entry point to the server.
- ☐ Change the value of the `"script"` field from

```
{ "test": "echo \"Error: no test specified\" && exit 1" },
```

 (line 6) to

```
{ "start": "nodemon server.js" }
```

. This specifies which file to run to start the server.
- ☐ After the `"main"` field (line 5), add the line: `"type": "module",`.
- 💡 Setting the `"type"` value in the `package.json` file to `"module"` tells Node.js to interpret

`.js` files using the newer ES module syntax.

☐ Move the `.gitignore` file located in the `client` directory to the root (`unearthed`) directory.

☐ In the `client` directory, delete the following files:

☐ `counter.js`

☐ `javascript.svg`

☐ `main.js`

☐ `public\vite.svg`

▼ 🔴 **Checkpoint 0:** Your `server\package.json` file should look like this 📌

```
{
  "name": "unearthed",
  "version": "1.0.0",
  "description": "",
  "main": "server.js",
  "type": "module",
  "scripts": {
    "start": "nodemon server.js"
  },
  "author": "",
  "license": "ISC",
  "dependencies": {
    "express": "^4.18.2",
    "nodemon": "^2.0.22"
  }
}
```

Step 1: Set up the file structure

In this step, we'll create directories to help keep our project organized and to make locating files easier as we move through the lab instructions.

☐ Inside the `client\public` directory, create a directory called `scripts` to store JavaScript files to dynamically add HTML elements to the DOM.

Inside the `server` directory, create the following directories:

☐ `data` to store the data we will display on the website.

☐ `routes` to store JavaScript files that define the routes for the API.

🔴 **Checkpoint 1:** Your repository should have the following file structure:

```
unearthed
├─ client
│   ├── public
│   │   └─ scripts
│   ├── index.html
│   ├── package.json
│   └─ style.css
├─ server
│   ├── data
│   ├── routes
│   └─ package.json
└─ .gitignore
```

Step 2: Set up the Express server

In this step, we'll create and connect to the Express server for your project.

- ☐ In the `server` directory, create a JavaScript file called `server.js`.

In the `server.js` file:

- ☐ Import `express`.

```
import express from 'express'
```

- ☐ Initialize the Express app.

```
const app = express()
```

To serve static files, such as images, HTML pages, and JavaScript files, we need to set up the `express.static` built-in middleware function in Express. The general syntax is `app.use(<insert middleware>)`.

✨ AI Opportunity

- ▼ Use AI to explain unfamiliar terms → "middleware"

What exactly *is* middleware? Ask Copilot what Express middleware is and for some examples. Feel free to ask any additional questions you have about its explanations or examples so you feel confident about the concept.

The results from ChatGPT will be slightly different, so if you don't have access to the Copilot Chat, ask the same questions in ChatGPT.

- ☐ To serve the files from the `client\public` directory, define a middleware function to serve static files from the `public` directory.

```
app.use('/public', express.static('./public'))
```

✨ AI Opportunity

▼ Use AI to explain code concepts

Why do we need to do this? Highlight and right-click on the code and select **Copilot > Explain this** to have it explain the purpose of this code.

If you don't have access to the Copilot Chat, copy and paste the code into ChatGPT, then ask it to explain the code.

- ☐ To serve the files from the `client\public\scripts` directory, define a middleware function to serve static files from the `scripts` directory.

```
app.use('/scripts', express.static('./public/scripts'))
```

- ☐ Define a route for the root URL of the server with parameters `req` and `res`. This will help us quickly see that our server is working and responding correctly when we start it.

```
app.get('/', (req, res) => {  
  res.status(200).send('<h1 style="text-align: center; margin-top: 50px;">UnEarthed API</h1>')  
})
```

✨ AI Opportunity

▼ Use AI to understand provided code → Status 200

In this code, we're responding with the status 200. If you don't remember what the status 200 means, feel free to ask Copilot to explain. You might also ask about other status codes that are used and examples of when they're used.

If you don't have access to the Copilot Chat, copy and paste the code into ChatGPT and ask it to explain the status 200.

- ☐ Start a server on port `process.env.PORT` or `3001`.

```
const PORT = process.env.PORT || 3001

app.listen(PORT, () => {
  console.log(`🚀 Server listening on http://localhost:${PORT}`)
})
```

💡 The `process.env.PORT` refers to a `PORT` environment variable. In this code segment, the server will first check to see if there is a `PORT` environment variable and use the `3001` port otherwise. In a development environment, it's easy to have your server listen on a hardcoded port, like 3001. But in many production environments, the server doesn't get to decide which port it listens on -- instead, it's assigned a port by the environment, and the server needs to be able to find out what that port is and use it.

Now let's try running the server!

- ☐ In the terminal, open a new terminal window or tab.
- ☐ Navigate to the `server` directory.
- ☐ Run the command `npm start`.

📌 **Checkpoint 2:** When you run the server, you should see `UnEarthed API` displayed.

UnEarthed API

► Your `server/server.js` file should look like this 📌

Step 3: Make an endpoint to retrieve the gift data

In this step, we'll load our gift data and set up a route to access this data.

- ☐ In the `server/data` directory, create a JavaScript file called `gifts.js`.
- ☐ Now it's time to populate our web app with actual data. In `gifts.js`, create a variable called `giftData` that is set to a JSON containing entries with the following fields:
 - ☐ `id`
 - ☐ `name`
 - ☐ `pricePoint`
 - ☐ `audience`
 - ☐ `image`
 - ☐ `description`
 - ☐ `submittedBy`
 - ☐ `submittedOn`

✨ AI Opportunity

▼ Use AI to generate sample data

To save some time, you could also ask Copilot to generate some sample data to use if you choose. In this case, you might structure your prompt something like:

You:

I need to create a JSON with fields id, name, pricePoint, audience, image, description, submittedBy, and submittedOn. Create a JSON with some sample data with these fields. For example:

```
"id": 1, "name": "Disco Ball Candle", "pricePoint": "$", "audience": "Candle Lovers",  
"image": "https://tinyurl.com/39xc9h6y", "description": "A party in a candle",  
"submittedBy": "Sasha", "submittedOn": "2022-09-04T14:48:00"
```

Keep in mind, though, that it likely won't retrieve valid URLs for the image. You should double-check the URLs it provides or replace them with your own.

If you don't have access to Copilot Chat, you can also ask ChatGPT the same question.

▼ Want to use the data we used in the exemplar? Copy and paste our sample data

```
const giftData = [  
  {  
    "id": 1,  
    "name": "Disco Ball Candle",  
    "pricePoint": "$",  
    "audience": "Candle Lovers",  
    "image": "https://tinyurl.com/39xc9h6y",  
    "description": "A party in a candle",  
    "submittedBy": "Sasha",  
    "submittedOn": "2022-09-04T14:48:00"  
  },  
  {  
    "id": 2,  
    "name": "Lego Flower Bouquet Kit",  
    "pricePoint": "$",  
    "audience": "Green Thumbs",  
    "image": "https://tinyurl.com/ydpv2bt9",  
    "description": "A gift and fun activity to do together",  
    "submittedBy": "Isabel",  
    "submittedOn": "2023-02-14T12:32:00"  
  },  
  {  

```

```
    "id": 3,
    "name": "Nintendo Switch Lite",
    "pricePoint": "$$",
    "audience": "Gamers",
    "image": "https://tinyurl.com/2cdf59x",
    "description": "A gift for gamers on-the-go",
    "submittedBy": "Mario",
    "submittedOn": "2022-09-13T11:01:00"
  },
  {
    "id": 4,
    "name": "Nike Panda Dunks",
    "pricePoint": "$$",
    "audience": "Sneakerheads",
    "image": "https://tinyurl.com/y9bzv8xd",
    "description": "A fan-favorite black and white color-blocking sneaker",
    "submittedBy": "Jason",
    "submittedOn": "2023-03-10T05:23:00"
  },
  {
    "id": 5,
    "name": "Razer Kitty Headset",
    "pricePoint": "$$",
    "audience": "Gamers",
    "image": "https://tinyurl.com/4pt84wry",
    "description": "Create the purrfect gaming look with your own killer kitty style",
    "submittedBy": "Pixie",
    "submittedOn": "2011-10-10T14:48:00"
  },
  {
    "id": 6,
    "name": "Nellsie Sunset Lamp Projection",
    "pricePoint": "$",
    "audience": "Sungazers",
    "image": "https://tinyurl.com/y76zsc37",
    "description": "Sunshine any time, any place",
    "submittedBy": "Melody",
    "submittedOn": "2011-10-10T14:48:00"
  },
  {
    "id": 7,
    "name": "Frigidaire Retro Mini Fridge",
    "pricePoint": "$",
    "audience": "Foodies",
    "image": "https://tinyurl.com/dzyhec8a",
    "description": "A retro way of keeping things cool",
    "submittedBy": "Grover",
    "submittedOn": "2023-01-01T14:48:00"
  },
  {
    "id": 8,
    "name": "Crosley Discover Turntable",
    "pricePoint": "$",
```

```

    "audience": "Music Lovers",
    "image": "https://tinyurl.com/4vycxmkm",
    "description": "A portable turntable that is ready to play right out of the box",
    "submittedBy": "Marvin",
    "submittedOn": "2023-04-01T07:02:00"
  },
  {
    "id": 9,
    "name": "Brandon Blackwood Parker Bag",
    "pricePoint": "$$",
    "audience": "Fashionistas",
    "image": "https://tinyurl.com/2n6pszw7",
    "description": "A perfect pop of color",
    "submittedBy": "Brandon",
    "submittedOn": "2023-01-10T14:48:00"
  }
]

export default giftData

```

Next, let's create a route that sends the provided data when requested.

- ☐ In the `routes` directory, create a JavaScript file called `gifts.js`

In the `routes/gifts.js` file:

- ☐ In the `routes/gifts.js` file, import `express` from `express` and `path` from the `path` package.

```

import express from 'express'
import path from 'path'

```

💡 The `path` module provides utilities for working with file and directory paths. It can be used to manipulate file paths, normalize them, join them together, and extract information such as the directory name, file name, and file extension.

- ☐ Import the `fileURLToPath` method from the `url` module.

```

import { fileURLToPath } from 'url'

```

💡 The `fileURLToPath` function is a utility function that converts a file URL to a file path. It takes a single argument, which is a file URL in the form of a string, and returns a file path in the form of a string. This function is useful when working with file URLs in Node.js, as many Node.js APIs expect file paths rather than file URLs.

- ☐ Import `giftData` from the `data/gift.js`.

```
import giftData from '../data/gifts.js'
```

- ☐ Convert the `import.meta.url` property to a file path and store the result in a variable called `__filename`.
- ☐ Create a variable called `__dirname` to contain the directory name of the current module file.

```
const __filename = fileURLToPath(import.meta.url)
const __dirname = path.dirname(__filename)
```

💡 The `import.meta.url` is a special property that contains the URL of the current module file. This URL can be used to determine the file path of the module file using the `fileURLToPath` function.

✨ AI Opportunity

▼ Use AI to explain code concepts

Why do we need this code? Highlight and right-click on the code and choose **Copilot > Explain this** to get an explanation. Feel free to ask additional follow-up questions to help you understand its purpose and necessity.

If you don't have access to Copilot Chat, copy and paste the code into ChatGPT so you can ask it for an explanation.

- ☐ Create an Express router.

```
const router = express.Router()
```

- ☐ Create a `GET` route at the `/` endpoint that responds with status `200` and sends a JSON of the `giftData`.

```
router.get('/', (req, res) => {
  res.status(200).json(giftData)
})
```

- ☐ Create another `GET` route at the `/:giftId` endpoint that responds with status `200` and sends the `gift.html` file (we'll be creating this later).

```
router.get('/:giftId', (req, res) => {  
  res.status(200).sendFile(path.resolve(__dirname, '../public/gift.html'))  
})
```

- ☐ Export the router.

```
export default router
```

Next, we need to modify the `src/server.js` to use this endpoint as a route.

- ☐ Import the router from `routes/gifts.js`.

```
import giftsRouter from './routes/gifts.js'
```

- ☐ Add the `/gifts` endpoint to the app by adding the following code after the middleware configuration.

```
app.use('/gifts', giftsRouter)
```

📌 **Checkpoint 3:** The endpoint `/gifts` should now return all the gifts data from the provided JavaScript file.

```
[{"id":1,"name":"Disco Ball Candle","pricePoint":"$","audience":"Candle Lovers","image":"https://tinyurl.com/39xc9h6y","description":"A party in a candle","submittedBy":"Sasha","submittedOn":"2022-09-04T14:48:00"}, {"id":2,"name":"Lego Flower Bouquet Kit","pricePoint":"$","audience":"Green Thumbs","image":"https://www.lego.com/cdn/cs/set/assets/blt74fd1228316303aa/10280_alt1.jpg","description":"A gift and fun activity to do together","submittedBy":"Isabel","submittedOn":"2023-02-14T12:32:00"}, {"id":3,"name":"Nintendo Switch Lite","pricePoint":"$$","audience":"Gamers","image":"https://tinyurl.com/2cdf59x","description":"A gift for gamers on-the-go","submittedBy":"Mario","submittedOn":"2022-09-13T11:01:00"}, {"id":4,"name":"Nike Panda Dunks","pricePoint":"$$","audience":"Sneakerheads","image":"https://tinyurl.com/y9bzv8xd","description":"A fan-favorite black and white color-blocking sneaker","submittedBy":"Jason","submittedOn":"2023-03-10T05:23:00"}, {"id":5,"name":"Razer Kitty Headset","pricePoint":"$$","audience":"Gamers","image":"https://cdn.shopify.com/s/files/1/0518/1469/5101/products/product-image-1650889788.jpg","description":"Create the purrfect gaming look with your own killer kitty style","submittedBy":"Pixie","submittedOn":"2011-10-10T14:48:00"}, {"id":6,"name":"Nellsie Sunset Lamp Projection","pricePoint":"$","audience":"Sungazers","image":"https://tinyurl.com/y76zsc37","description":"Sunshine any time, any place","submittedBy":"Melody","submittedOn":"2011-10-10T14:48:00"}, {"id":7,"name":"Frigidaire Retro Mini Fridge","pricePoint":"$","audience":"Foodies","image":"https://tinyurl.com/dzyhec8a","description":"A retro way of keeping things cool","submittedBy":"Grover","submittedOn":"2023-01-01T14:48:00"}, {"id":8,"name":"Crosley Discover Turntable","pricePoint":"$","audience":"Music Lovers","image":"https://tinyurl.com/4vyxmkkm","description":"A portable turntable that is ready to play right out of the box","submittedBy":"Marvin","submittedOn":"2023-04-01T07:02:00"}, {"id":9,"name":"Brandon Blackwood Parker Bag","pricePoint":"$$","audience":"Fashionistas","image":"https://tinyurl.com/2n6pszw7","description":"A perfect pop of color","submittedBy":"Brandon","submittedOn":"2023-01-10T14:48:00"}]
```

Step 4: Set up the home page

- ☐ In the `client\public` directory, download and add [this Unearthed logo](#), or you can find or create your own logo to use in your web app instead. Name the image `logo.png`
- ☐ In the `client` directory, modify the `index.html` code to link to your stylesheet.

We will use DOM manipulation to add child elements to `header` and `main` tags. Therefore, we need to add and link JavaScript files to `index.html`.

- ☐ In the `public/scripts` directory, create the following JavaScript files:
 - ☐ `header.js` to contain all the JavaScript code to add elements to the `<header>` element.
 - ☐ `gifts.js` to contain contain all the JavaScript code to add elements to the `<main>` element.
- ☐ Back in your `index.html` file, link `header.js` and `gifts.js` in the `head` tag.

✨ **AI Opportunity**

▼ Use AI to brainstorm ideas for your code → Boilerplate code

You can also ask ChatGPT to generate this code for you. Make sure to be specific about what you need to do and the steps you expect it to perform. Review the code it generates to check that it makes sense before you add it to your own code!

▼ Want to double-check your code? Compare what you have to this 

```
<!DOCTYPE html>

<head>
  <title>UnEarthed</title>
  <link rel="icon" type="image/x-icon" href="/logo.png">
  <link rel="stylesheet" href="/style.css">
  <script defer src="/scripts/header.js"></script>
  <script defer src="/scripts/gifts.js"></script>
</head>

<body>
  <header></header>
  <main id="main-content"></main>
</body>

</html>
```

Windows users only: Before running the next command, you may need to update your PowerShell execution policy. If you see an error like *"running scripts is disabled on this system"* when running `npm run dev` below, open PowerShell as Administrator and run:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned -Force
```

You only need to do this once on your machine.

- ☐ Open a new terminal and navigate to the `client` directory. Run the command `npm run build` then `npm run dev` to test your web app.

🔴 **Checkpoint 4:** You should see a blank page loaded when you navigate to the frontend. Check the console to make sure you don't have any errors.

Step 5: Create a header using JavaScript

In this step, we'll use JavaScript to manipulate the DOM to add elements that will make up the header.

In the `scripts/header.js` file:

- ☐ Create a variable called `header` that points to the `header` tag.

```
const header = document.querySelector('header')
```

- ☐ Create a `div` element with a class name `header-container`.

```
const headerContainer = document.createElement('div')
headerContainer.className = 'header-container'
```

- ☐ Create a `div` element with a class name `header-left`.

✨ AI Opportunity

▼ Use AI to brainstorm ideas for your code → Generating example code

We need to do this in the same way that we created the previous `div` element. If you need some help, ask Copilot to use the previous `div` element as an example and tell it that you need another `div` element with a class name `header-left`. For example, you might structure your prompt like this:

You:

My code currently creates a div element with the class name header-container. I need to create another div element with the class name header-left using the same structure. Can you show me how I should do this?

If you don't have access to Copilot Chat, copy and paste the code you have so far into ChatGPT so you can ask it the same question.

▼ Want to double-check your code? Compare what you have to this

```
const headerLeft = document.createElement('div')
headerLeft.className = 'header-left'
```

- ☐ Create an `img` element and assign its source to `logo.png`.

✨ AI Opportunity

▼ Use AI to brainstorm ideas for your code → Generating example code

This one is similar to the `div` elements, except this time we need to set the `src` for the `img` element instead of the class name. If you're not sure how to do this, ask Copilot for an example that you can use to help you. For example:

You:

I need to create an `img` element and assign its `src` to the `logo.png` image that I have.
Can you give me an example of how I would do this?

If you don't have access to Copilot Chat, copy and paste your code into ChatGPT, then ask it the same question.

▼ Want to double-check your code? Compare what you have to this ↴

```
const headerLogo = document.createElement('img')
headerLogo.src = '/logo.png'
```

☐ Create a `h1` element and set its text content to `UnEarthed`.

▼ Want to double-check your code? Compare what you have to this ↴

```
const headerTitle = document.createElement('h1')
headerTitle.textContent = 'UnEarthed'
```

☐ Append the logo and the title to the `div` with the class name `header-left`.

▼ Want to double-check your code? Compare what you have to this ↴

```
headerLeft.appendChild(headerLogo)
headerLeft.appendChild(headerTitle)
```

- ☐ Create a `div` element with a class name `header-right`.

▼ Want to double-check your code? Compare what you have to this 

```
const headerRight = document.createElement('div')
headerRight.className = 'header-right'
```

- ☐ Create a `button` element and set its text content to `Home`. Register a `click` event listener to the button that redirects the window to the root page.

```
const headerButton = document.createElement('button')
headerButton.textContent = 'Home'

headerButton.addEventListener('click', function handleClick(event) {
  window.location = '/'
})
```

AI Opportunity

▼ Use AI to understand provided code

Have questions about this code? Highlight and right-click on the code and choose **Copilot > Explain this** for an explanation.

Alternatively, if you don't have Copilot Chat, copy and paste the code into ChatGPT, and ask it to explain the code.

- ☐ Append the button to the `div` with the class name `header-right`.

▼ Want to double-check your code? Compare what you have to this 

```
headerRight.appendChild(headerButton)
```

- ☐ Append the left and right header `div` elements to the `div` with the class name `header-container`.

▼ Want to double-check your code? Compare what you have to this ↻

```
headerContainer.appendChild(headerLeft)
headerContainer.appendChild(headerRight)
```

☐ Append the `header-container` element to the `header` element.

▼ Want to double-check your code? Compare what you have to this ↻

```
header.appendChild(headerContainer)
```

☐ Open a new terminal and navigate to the `client` directory. Run the command `npm run dev` to test your web app.

📌 **Checkpoint 5:** Your home page should now have a header that allows the user to click to see the homepage should they navigate to another page.



Step 6: Dynamically display gift data on the home page

In this step, we'll retrieve the gift data from the server and display it on our website.

- ☐ In the `client` directory, create a new file called `vite.config.js`.

- ☐ In this file, copy and paste the following code:

```
import { defineConfig } from 'vite'

export default defineConfig({
  build: {
    outDir: '../server/public',
    emptyOutDir: true
  },
  server: {
    proxy: {
      '/gifts': {
        target: 'http://localhost:3001'
      }
    }
  }
})
```

 The `vite.config.js` file is used to customize the behavior of the Vite tool. You can define various options, such as development server options and production build options.

In this code segment, we are specifying that when we run the command `npm run build`, the resources in the `public` directory will be output to the `server/public` directory. We are also specifying that when we try to access the `/gifts` endpoint from the client, we want the server to access this route at `http://localhost:3001` while we are in development.

Feel free to ask Copilot or ChatGPT about the `vite.config.js` file to learn more about what can be configured here or to ask questions about these configurations.

Now, let's retrieve the data from the `/gifts` endpoint and display the data on the website.

- ☐ In `public/scripts/gifts.js`, create an asynchronous function called `renderGifts()`.

```
const renderGifts = async () => {
}
```

- ☐ In the body of the `renderGifts()` function, use `fetch()` to retrieve the gift data at the `/gifts` endpoint, then parse the response as a JSON.

```
const response = await fetch('/gifts')
const data = await response.json()
```

- ☐ Create a variable called `mainContent` that points to the element with the ID `main-content`.

```
const mainContent = document.getElementById('main-content')
```

- ☐ Create an `if/else` statement to use conditional rendering based on whether or not `data` is `null`.

```
if (data) {  
  
}  
else {  
  
}
```

- ☐ In the `else` branch, write JavaScript code that adds an `h2` element with the text `No Gifts Available 🙄` to `mainContent`.

✨ AI Opportunity

▼ Use AI as a pair programming partner

Try using Copilot or ChatGPT as a pair programmer for this if you need some help. Tell it to act as the driver, then give it the task it needs to complete one step at a time. For instance:

You:

You are an experienced software engineer. You and I will be pair programming on a project. I will be the navigator, and you will be the driver. As a navigator, when I explain the logic to you, I want you (as the driver) to implement it in JavaScript.

Copilot will respond confirming its role and what it should do. From here, you can then give it the logic you want it to implement. Give it one small step at a time, like:

You:

I would like you to create an `h2` element and assign it to a variable called `message`.

▼ Want to double-check your code? Compare what you have to this ↻

```
const message = document.createElement('h2')  
message.textContent = 'No Gifts Available 🙄'  
mainContent.appendChild(message)
```

- ☐ In the `if` branch, use `map()` to add an HTML card to the DOM for each gift in the data.

```
data.map(gift => {  
  
})
```

- ☐ For each `gift`, create a `div` with the class name `card`.

✨ AI Opportunity

▼ Use AI as a pair programming partner

Try using Copilot or ChatGPT as a pair programmer for this if you need some help. Tell it to act as the driver, then give it the task it needs to complete one step at a time. For instance:

You:

You are an experienced software engineer. You and I will be pair programming on a project. I will be the navigator, and you will be the driver. As a navigator, when I explain the logic to you, I want you (as the driver) to implement it in JavaScript.

Copilot will respond confirming its role and what it should do. From here, you can then give it the logic you want it to implement. Give it one small step at a time, like:

You:

Inside my `data.map()` code, I would like you to create a `div` element with the class name `card`.

▼ Want to double-check your code? Compare what you have to this ↻

```
const card = document.createElement('div')  
card.classList.add('card')
```

- ☐ Create another `div` with the class name `top-container`.

✨ AI Opportunity

▼ Use AI as a pair programming partner

If you started a pair programming conversation with Copilot earlier, you can continue the conversation here to add this `div` element:

You:

Next, I would like you to create another div element for each gift and assign it to a variable called `topContainer`. The div element should also be given the class name `top-container`.

▼ Want to double-check your code? Compare what you have to this ↴

```
const topContainer = document.createElement('div')
topContainer.classList.add('top-container')
```

☐ Create another `div` with the class name `bottom-container`.

▼ Want to double-check your code? Compare what you have to this ↴

```
const bottomContainer = document.createElement('div')
bottomContainer.classList.add('bottom-container')
```

☐ Set the background image of the `top-container` element to the gift's image.

▼ Want to double-check your code? Compare what you have to this ↴

```
topContainer.style.backgroundImage = `url(${gift.image})`
```

☐ Create an `h3` element, and set its text content to the gift's name. Then, append it to the `bottom-container` element.

▼ Want to double-check your code? Compare what you have to this ↴

```
const name = document.createElement('h3')
name.textContent = gift.name
bottomContainer.appendChild(name)
```

- ☐ Create a `p` element, and set its text content to the gift's price. Then, append it to the `bottom-container` element.

▼ Want to double-check your code? Compare what you have to this 

```
const pricePoint = document.createElement('p')
pricePoint.textContent = 'Price: ' + gift.pricePoint
bottomContainer.appendChild(pricePoint)
```

- ☐ Create another `p` element, and set its text content to the gift's audience. Then, append it to the `bottom-container` element.

▼ Want to double-check your code? Compare what you have to this 

```
const audience = document.createElement('p')
audience.textContent = 'Great For: ' + gift.audience
bottomContainer.appendChild(audience)
```

- ☐ Create an `a` element, and set its text content to `Read More >`. Set the `href` of the link to `/gifts/gift.id`. Set the `role` to `button`. Then, append it to the `bottom-container` element.

AI Opportunity

▼ Use AI as a pair programming partner

Use the strategies you've learned for working with Copilot or ChatGPT to generate this code. If you're using it as a pair programmer, be sure to ask for one step at a time to improve the accuracy of the code it suggests or decompose the task into small steps. For example:

You:

Write JavaScript code that does the following:

1. Create an a element
2. Set its text content to "Read More >"
3. Set the href of the link to /gifts/gift.id
4. Set the role to button
5. Append the a element to the bottom-container element

▼ Want to double-check your code? Compare what you have to this ↴

```
const link = document.createElement('a')
link.textContent = 'Read More >'
link.setAttribute('role', 'button')
link.href = `/gifts/${gift.id}`
bottomContainer.appendChild(link)
```

- ☐ Append the `top-container` and `bottom-container` elements to the card.

▼ Want to double-check your code? Compare what you have to this ↴

```
card.appendChild(topContainer)
card.appendChild(bottomContainer)
```

- ☐ Append the `card` element to the `mainContent`.

▼ Want to double-check your code? Compare what you have to this ↴

```
mainContent.appendChild(card)
```

- ☐ At the end of the file (outside of the `renderGifts()` function definition), call the `renderGifts()` function.
- ☐ Open a new terminal and navigate to the `client` directory. Run the command `npm run build` then `npm run dev` to test your web app.

📌 **Checkpoint 6:** Your app should now have a basic list of elements on the home page without styling. You'll make these into cards in the next step!

UnEarthed

[Home](#)

Disco Ball Candle

Price: \$

Great For: Candle Lovers

[Read More →](#)

Lego Flower Bouquet Kit

Price: \$

Great For: Green Thumbs

[Read More →](#)

Nintendo Switch Lite

Price: \$\$

Great For: Gamers

[Read More →](#)

Nike Panda Dunks

Price: \$\$

Great For: Sneakerheads

[Read More →](#)

Razer Kitty Headset

Price: \$\$

Great For: Gamers

► Your `scripts\gifts.js` file should look like this 🖱️

Step 7: Add styling

It is time to style the frontend!

- ☐ Replace the code in the `style.css` file in the `client` directory with the code below. Feel free to use this template and modify the styling as you'd like.

► Code for `style.css` here [🔗](#) [🔗](#) [🔗](#)

✨ AI Opportunity

▼ *Use AI to understand provided code*

If you have questions about any of this CSS copy, highlight and right-click the code and choose **Copilot > Explain this** or copy and paste it into ChatGPT and ask it to explain the code.

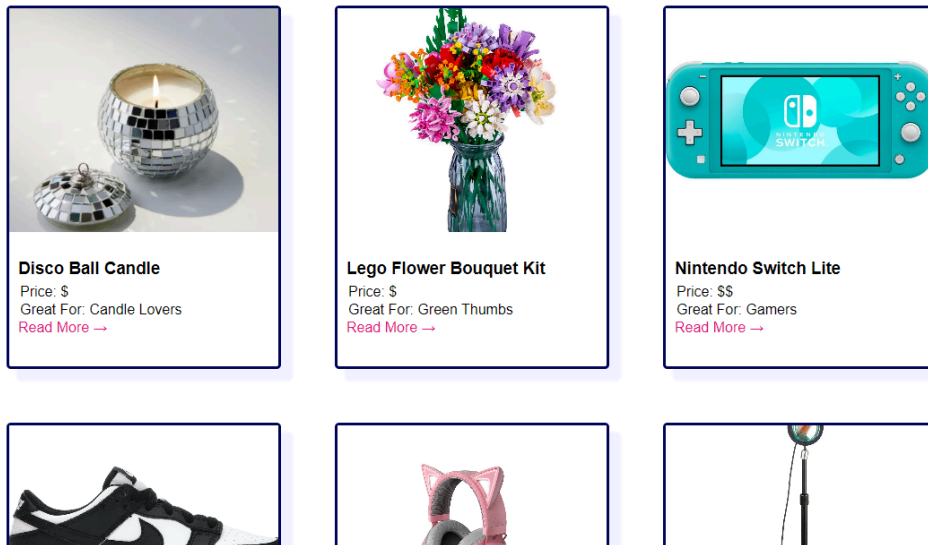
You can also ask Copilot or ChatGPT to suggest modifications for any parts of the styling you want to change.

- ☐ Open a new terminal and navigate to the `client` directory. Run the command `npm run build` then `npm run dev` to test your web app.

📌 **Checkpoint 7:** Your app's home page is looking good now!



HOME



Step 8: Create the gift detail page

We want the user to be able to view the details for an individual gift when they click the "Read More" link. Let's set up our gift detail page next.

- ☐ In the `public` directory, create an HTML page called `gift.html`.
- ☐ In the `gift.html` file, add HTML boilerplate code that has empty `head` and `body` elements.
- ☐ Inside the `head` tag, add a title and link the stylesheet, `header.js` file and `gift.js` file.
- ☐ Inside the `body` tag, add an empty `header` element.

```
<header></header>
```

- ☐ After the empty `header` element, add a `main` element with the ID `gift-content` and class name `gift-info`.

```
<main id="gift-content" class="gift-info"></main>
```

Inside the `main` element:

- ☐ Create a `div` with the class name `image-container`.
- ☐ Inside the `image-container` element, create an `img` element with the ID `image` and the `src` set to `""`.
- ☐ Create a `div` with the class name `gift-details`.
- ☐ Inside the `gift-details` element:
 - ☐ Add an `h2` element with an `id` called `name`.
 - ☐ Add a `p` element with an `id` called `submittedBy`.
 - ☐ Add a `p` element with an `id` called `submittedOn`.
 - ☐ Add a `p` element with an `id` called `pricePoint`.
 - ☐ Add a `p` element with an `id` called `audience`.
 - ☐ Add a `p` element with an `id` called `description`.

✨ AI Opportunity

▼ Use AI to brainstorm ideas for your code → Suggesting code

Use the strategies you've learned for working with Copilot or ChatGPT to have it generate this code for you if you'd like. Remember, give it one step at a time to complete! Review the code it suggests to make sure it achieves the goal before adding it to your own code.

🔴 **Checkpoint 8:** Your app can now access the `gift.html` page when we click the "Read More" link on a gift, but it doesn't show anything...yet!

Step 9: Dynamically display gift details

In this step, we'll use JavaScript to dynamically display the gift details.

- ☐ In the `scripts/gift.js` file, create an asynchronous function called `renderGift()`.

```
const renderGift = async () => {  
  
}
```

- ☐ In the body of the `renderGift()` function, parse the ID as an `int` from the URL.

```
const requestedID = parseInt(window.location.href.split('/').pop())
```

- ☐ Using `fetch()`, fetch the gift data using the `/gifts` endpoint and parse the response as a JSON.

✨ AI Opportunity

▼ Use AI to explain code concepts → `fetch()`

Forget how? Ask Copilot for an example using `fetch()` to retrieve data from an API and use that to help you write your code. If you're still stuck, tell it what you are trying to do.

If you don't have access to Copilot Chat, copy and paste your code into ChatGPT so it has the appropriate context to help you do this.

▼ Want to double-check your code? Compare what you have to this ↻

```
const response = await fetch('/gifts')
const data = await response.json()
```

- ☐ Create a variable called `giftContent` that points to the element with the ID `gift-content`.

```
const giftContent = document.getElementById('gift-content')
```

- ☐ Create a variable called `gift`.

```
let gift
```

- ☐ Use an `if` statement to check if `data` is not `null`. If so, find the gift in the gift data that has the same ID as the requested ID.

```
gift = data.find(gift => gift.id === requestedID)
```

- ☐ Create an `if/else` statement to conditionally render the gift data based on whether or not `gift` is `null`.

```
if (gift) {  
  
}  
else {  
  
}
```

- ☐ In the `else` branch, write JavaScript code that adds an `h2` element with text `No Gifts Available 🙄` to `giftContent`.

▼ Want to double-check your code? Compare what you have to this 

```
const message = document.createElement('h2')  
message.textContent = 'No Gifts Available 🙄'  
giftContent.appendChild(message)
```

In the `if` branch:

- ☐ Set the `src` of the `image` element to the matching gift's image.
- ☐ Set the text content of the element with the ID `name` to matching gift's name.
- ☐ Set the text content of the element with the ID `submittedBy` to the matching gift's submitted by date.
- ☐ Set the text content of the element with the ID `pricePoint` to the matching gift's price point.
- ☐ Set the text content of the element with the ID `audience` to the matching gift's audience.
- ☐ Set the text content of the element with the ID `description` to the matching gift's description.
- ☐ Set the title of the page the gift's name.

✨ AI Opportunity

▼ Use AI to brainstorm ideas for your code → Suggesting code

Use the strategies you've learned for working with Copilot or ChatGPT to have it generate this code for you if you'd like. Remember, give it one step at a time to complete! Review the code it suggests to make sure it achieves the goal before adding it to your own code.

▼ Want to double-check your code? Compare what you have to this ↴

```
document.getElementById('image').src = gift.image
document.getElementById('name').textContent = gift.name
document.getElementById('submittedBy').textContent = 'Submitted by: ' + gift.submittedBy
document.getElementById('pricePoint').textContent = 'Price: ' + gift.pricePoint
document.getElementById('audience').textContent = 'Great For: ' + gift.audience
document.getElementById('description').textContent = gift.description
document.title = `UnEarthed - ${gift.name}`
```

- ☐ Call the `renderGift()` function at the end of the file.

```
renderGift()
```

- ☐ Open a new terminal and navigate to the `client` directory. Run the command `npm run build` then `npm run dev` to test your web app.

📌 **Checkpoint 9:** Your app now displays the gift's details when you click on the `"Read More"` link.



UnEarthed

[HOME](#)

Nintendo Switch Lite

Submitted by: Mario

Price: \$\$

Great For: Gamers

A gift for gamers on-the-go

Step 10: Set up up 404 page

- ☐ In the `public` directory, create an HTML document called `404.html`.

☐ In `404.html`, copy and paste the provided starter code below:

▼ Code for `404.html` here   

```
<!DOCTYPE html>
<head>
  <title>UnEarthed</title>
  <link rel="icon" type="image/x-icon" href="/public/logo.png">
  <link rel="stylesheet" href="../style.css">
</head>

<body>
  <main class="fof-container">
    <div class="fof">
      <h1>Error 404</h1>
      <a href="/"><p>Return Home</p></a>
    </div>
  </main>

</body>

</html>
```

We need to update our `gifts.js` to only generate the gifts data if the user is on the home page and show the 404 page if the user attempts to navigate to an invalid URL.

☐ In the `scripts/gifts.js` file, replace the call to the `renderGifts()` function with an `if/else` statement.

```
if () {
}
else {
}
```

☐ Before the `if/else` statement, create a variable called `requestedURL` that extracts the portion of the URL after the `/`.

✨ AI Opportunity

▼ Use AI to understand provided code → `renderGift()`

We did this earlier when we wrote the `renderGift()` function. If you're not sure which part of the `renderGift()` function did this, highlight and right-click on the code and choose **Copilot**

> **Start Code Chat** and ask it which line extracts the portion of the URL after the `/`.

If you don't have access to Copilot Chat, copy and paste your `renderGift()` code into ChatGPT, and ask it the same question.

▼ **Want to double-check your code? Compare what you have to this** ↻

```
const requestedUrl = window.location.href.split('/').pop()
```

- ☐ In the `if` statement, check if `requestedUrl` is not `null`. If it isn't `null` (there is something after the `/`), set the `window.location.href` to the `404.html` page.

```
if (requestedUrl) {  
  window.location.href = '../404.html'  
}  
else {  
  
}
```

- ☐ In the `else` statement, call the `renderGifts()` function.

```
else {  
  renderGifts()  
}
```

- ☐ Open a new terminal and navigate to the `client` directory. Run the command `npm run build` then `npm run dev` to test your web app.

📌 **Checkpoint 10:** Your app should now show the corresponding 404 page when the user attempts to navigate to a route that you have not yet defined.

Error 404

[Return Home](#)

   Congratulations, you've completed your first lab!   